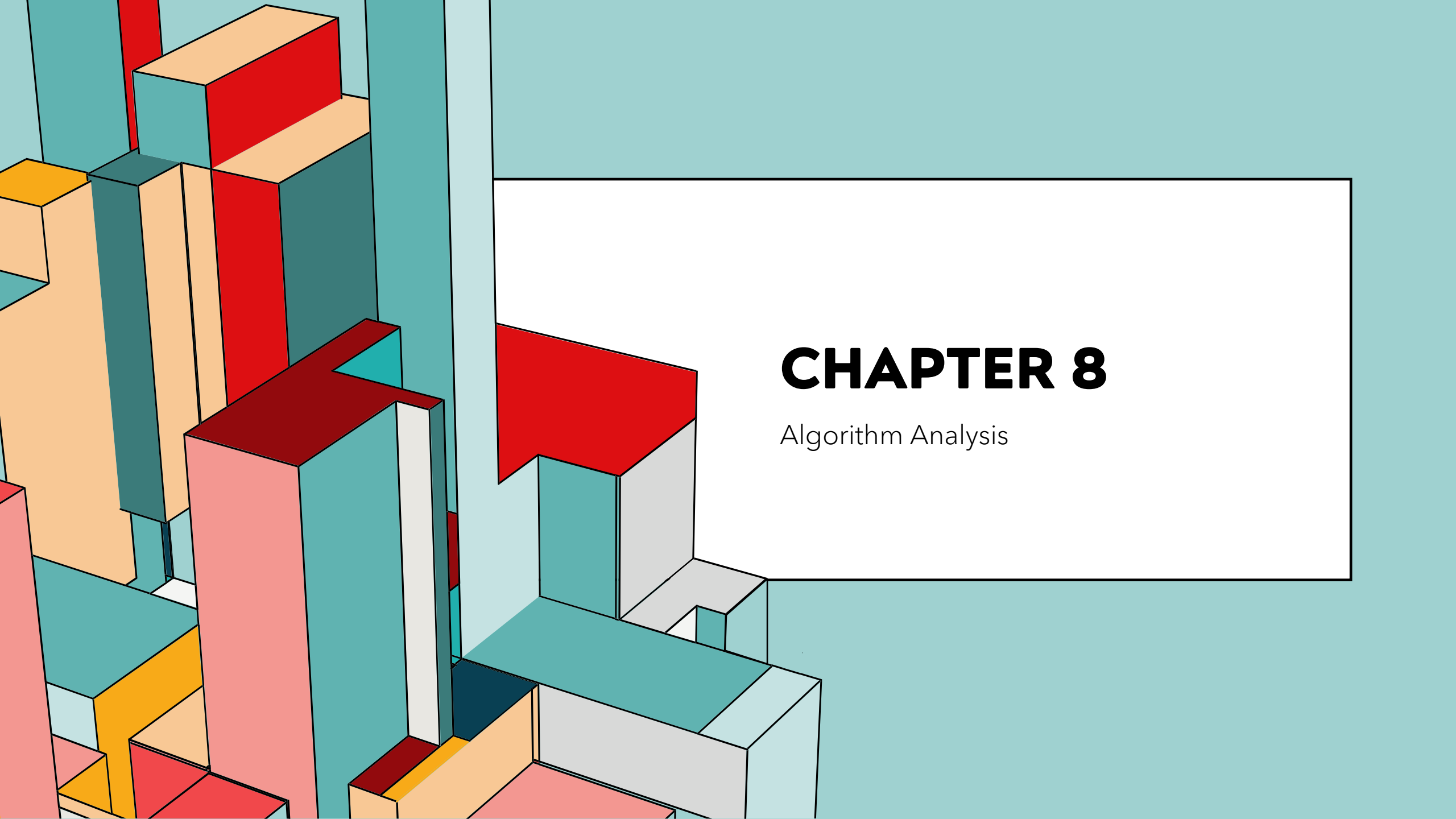




CHAPTER 8

Algorithm Analysis

อัลกอริทึม (ALGORITHM)

- ขั้นตอนวิธีที่ใช้ในการแก้ปัญหา (หาเส้นทางจาก Initial state -> End state)
- อัลกอริทึม เป็นลักษณะของแนวคิด ซึ่งสามารถแสดงออกมาได้หลายรูปแบบ
 - ชูโดโค้ด (Pseudo code)
 - แผนผัง (Flowchart)
 - โปรแกรม (Program)

ซูโดโค้ด (PSEUDO CODE)

```
Algorithm sample (pageNumber) #ชื่อไปเลย  
This algorithm reads a file and prints a report.  
Pre    pageNumber passed by reference  
Post   Report Printed  
        pageNumber contains number of pages in report  
Return Number of lines printed  
1 loop (not end of file)  
  1 read file  
  2 if (full page)  
    1 increment page number  
    2 write page heading  
  3 end if  
  4 write report line  
  5 increment line count  
2 end loop  
3 return line count  
end sample
```

- ① ทำงานเร็ว
- ② ประหยัดพื้นที่

การวิเคราะห์อัลกอริทึม

(ALGORITHM ANALYSIS)

- ปัญหาหนึ่งๆ สามารถถูกแก้ได้ด้วยหลากหลายวิธี (หลายอัลกอริทึม)
- เพื่อเลือกวิธีที่ดี เราต้องสามารถเปรียบเทียบประสิทธิภาพของอัลกอริทึมได้
- การวิเคราะห์อัลกอริทึม พิจารณา ↗ class นี้สนใจ time บ้างนะ
 - Time Complexity : เวลาที่ใช้ในการรันโปรแกรม # ยิ่งน้อยยิ่งดี
 - Space Complexity : พื้นที่หรือหน่วยความจำที่ใช้ในการรันโปรแกรม # คำนึงถึง ↗

ประสิทธิภาพของอัลกอริทึม $f(n)$

(ALGORITHM EFFICIENCY)

```
Algorithm A:  
  app-code  
  app-code  
  app-code  
end A
```

ขนาดข้อมูลที่มีผลต่อ ก. ทำงาน

ตัววัด
ประสิทธิภาพ

- พิจารณาประสิทธิภาพจากจำนวนของอิลิเมนต์ที่ถูกโปรเซส
- $f(n) = \text{efficiency}$
- ถ้าฟังก์ชันเป็น Linear (no loops)
 - Efficiency = จำนวนคำสั่งที่ทำงานทั้งหมด

LINEAR LOOPS

- มีการเพิ่ม/ลดค่าภายในลูปอย่างคงที่

$\oplus, \ominus$

```
for (i=0; i<1000; i++)
```

application code

```
end for
```

Iterations : 1000 รอบ รอบละ 1 คำสั่ง

```
for (i=0; i<1000; i+=2)
```

application code

```
end for
```

Iterations : 500 รอบ รอบละ 1 คำสั่ง

จาก input f

```
for (i=0; i<n; i++)
```

application code

```
end for
```

$f(n) = n$

```
for (i=0; i<n; i+=2)
```

application code

```
end for
```

$f(n) = \frac{n}{2}$

ถ้า $i+=3 \rightarrow f(n) = \frac{n}{3}$

LOGARITHMIC LOOPS

รอบ
1
2
3
4

i
1
2
4
8

- เพิ่ม/ลดค่าภายในลูปเป็นจำนวนเท่าตัวด้วยการคูณหรือหาร

```
for (i=1; i<1000; i*=2)
    application code
end for
```

10 รอบ

```
for (i=1000; i>1; i/=2)
    application code
end for
```

10 รอบ

Iterations : $\hookrightarrow \log_2 1000$

1000
500
250
125
62
31
15
8
4
2

```
for (i=1; i<n; i*=2)
    application code
end for
```

```
for (i=n; i>1; i/=2)
    application code
end for
```

$f(n) = \log_2 n$

ถ้า $i* = 3$

ถ้า $i / = 3$

$f(n) = \log_3 n$

16
32
64
128
256
512

LOGARITHMIC LOOPS (CONT.)

Multiply		Divide	
Iteration	Value of i	Iteration	Value of i
1	1	1	1000
2	2	2	500
3	4	3	250
4	8	4	125
5	16	5	62
6	32	6	31
7	64	7	15
8	128	8	7
9	256	9	3
10	512	10	1
(exit)	1024	(exit)	0

ลูปซ้อนลูป → อัมตะต่อกัน

NESTED LOOPS

- เป็นลักษณะของลูปซ้อนลูป
- Iterations = outer loop iterations * inner loop iterations
- Linear Logarithmic

10 รอบ

```
for (i=0; i<10; i++)(linear)  
  for (j=1; j<10; j*=2) → Log  
    application code  
  end for  
end for
```

$\log_2 10 = 4 \text{ รอบ}$

จำนวน ลูป นอก * ลูป ใน
 $= 10 \log_2 10$

Iterations :

$$; f(n) = n \log_2 n$$

NESTED LOOPS (CONT.)

All Linear

- Quadratic

```
10 { for (i=0; i<10; i++)  
   { 10 { for (j=0; j<10; j++)  
       application code  
     }  
   }  
}
```

$$10 \times 10 = 100$$

Iterations :

$$; f(n) = n^2, n \times n$$

NESTED LOOPS (CONT.)

ตัวอย่าง Var ที่ขึ้นกับกัน

- Dependent Quadratic

```

for (i=1; i<=10; i++)
    for (j=0; j<=i; j++)
        application code
    end for
end for
    
```

Iterations :

j	i
0	1
1	x
2	2 x

i	สรุป
1	1 รอบ
2	2 รอบ
3	3 รอบ
⋮	⋮
⋮	⋮
⋮	⋮
10	10 รอบ

$$1 + 2 + 3 + 4 + \dots + 10 = 55$$

$$f(n) = \frac{n}{2} (n + 1)$$

Algorithm B :

$2 \text{ အိမ် } \left[\begin{array}{l} \text{app code} \\ \text{app code} \end{array} \right.$

$\frac{n}{4} \text{ လုပ်}$
 $\text{လုပ်တဲ့ } 2 \text{ အိမ်}$
 $= 2 \times \frac{n}{4} = \frac{n}{2}$

$\left[\begin{array}{l} \text{for } (i=0; i < n; i+=4); \\ \quad \text{app code} \\ \quad \text{app code} \\ \text{end for} \\ \text{app code} \end{array} \right] \rightarrow 1 \text{ အိမ်}$ $\nearrow \text{linear}$

end B

$$f(n) = \frac{n}{2} + 3$$

Problem 1

→ A : 220 คำสั่ง
→ B : 250 คำสั่ง

BIG-O NOTATION

- การวัดประสิทธิภาพเน้นที่ข้อมูลปริมาณมาก และสนใจค่าประมาณ (Approximation) มากกว่า ด้วยฟังก์ชันบิกโอ (Big-O) #จุดสนใจ.เก็บใส่
- ซึ่งเป็นการพิจารณาจาก อัตราการเติบโตของฟังก์ชัน -> ดูจากค่าที่มีผลกระทบมากที่สุดค่าเดียว

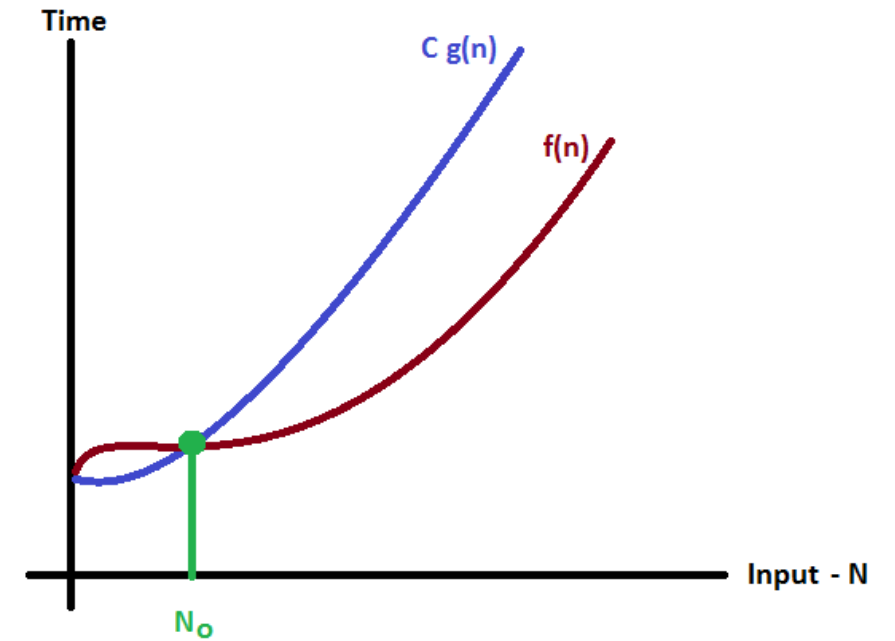
$$f(n) = 3n^2 + 2n + 5 \rightarrow \text{Big O} = O(n^2)$$

$$f(n) = 4n^3 + 2n \rightarrow \text{Big O} = O(n^3)$$

BIG-O NOTATION

- $f(n) = O(g(n))$ เมื่อ $f(n) \leq cg(n)$, for $c > 0$ และ $n \geq n_0 \geq 1$
- $f(n)$ เป็นบีกโอของ $g(n)$

$f(n)$ ที่ทำจะไม่ได้ก่อน Big O



problem 1

(A) $f(n) = 2n^3 + 100n + 10 \rightarrow O(n^3)$

(B) $f(n) = 50n^3 + 7 \rightarrow O(n^3)$

BIG-O NOTATION

① ถ้าฟังก์ชันมีลำดับที่ต่ำกว่าหรือเท่ากับลำดับที่สนใจ ให้ใช้ลำดับที่สนใจ

- พิจารณาแต่ละเทอมให้มีสัมประสิทธิ์เป็น 1

- สนใจเฉพาะเทอมที่ใหญ่ที่สุด

② ดูว่าลำดับที่สนใจมีกี่เทอม และในลำดับที่สนใจ

■ $\log n$ n $n \log n$ n^2 n^3 ... n^k 2^n $n!$ # ห้อย + มาก

$$f(n) = \frac{n}{2} + 3$$

$$\text{Big O} = 1n + 3$$

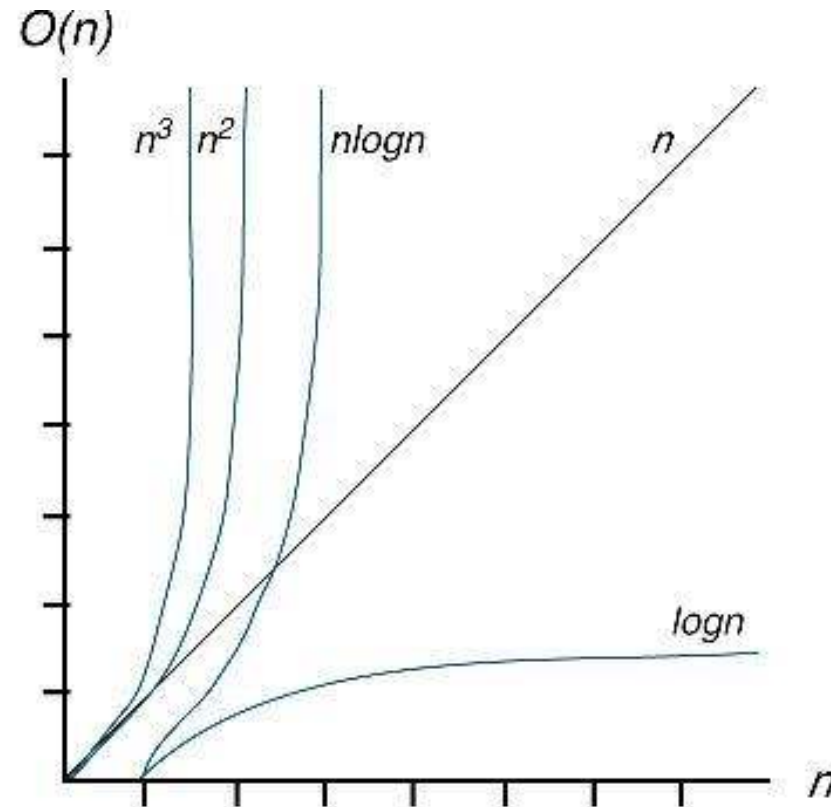
$$\approx n + 3$$

$$= O(n)$$

STANDARD MEASURES OF EFFICIENCY

N	N^2	N^3	2^N	$\log_2 N$	$N \log_2 N$
1	1	1	2	0	0
2	4	8	4	1	2
4	16	64	16	2	8
8	64	512	256	3	24
16	256	4096	65536	4	64
32	1024	32768	2147483648	5	160
64	4096	262144	?	6	384
128	16384	2097152	?	7	896
256	65536	16777216	?	8	2048

STANDARD MEASURES OF EFFICIENCY (CONT.)



STANDARD MEASURES OF EFFICIENCY (CONT.)

Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10,000^2$	minutes
Polynomial	$O(n^k)$	$10,000^k$	hours
Exponential	$O(c^n)$	$2^{10,000}$	intractable
Factorial	$O(n!)$	$10,000!$	intractable

Measures of Efficiency for $n = 10,000$

EXAMPLE

- $f(n) = 2n+7$
 - $f(n) = O(n)$
- $f(n) = n^3+3n-8$
 - $f(n) = O(n^3)$
- $f(n) = 1$ หรือ $f(n) = 100$
 - $f(n) = O(1)$

$f(n) = 1$
 $f(n) = 100$
 $f(n) = 500$

ประมาณภาพไว้กับ ค่าคงที่
ไม่ส่งผลต่อ

Algorithm C:

app-code
app-code] 2 คำสั่ง

for (i = 1; i < n; i *= 2):

app-code
app-code

for (j = 0; j < n; j += 3):

app-code
app-code

end for

app-code

end for

$\log_2 n$ รอบ

ของลูป

$2 + \frac{2n}{3} + 1$ คำสั่ง

$= \frac{2n}{3} + 3$ คำสั่ง

$\frac{2n}{3}$

คำสั่งที่ทำงานของ loop นอก = $\log_2 n \left(\frac{2n}{3} + 3 \right)$

→ หมายเหตุ *

$$f(n) = 2 + \log_2 n \left(\frac{2n}{3} + 3 \right)$$

$$= 2 + \frac{2}{3} n \log_2 n + 3 \log_2 n$$

$$= O(n \log n)$$

$\log_2 n$

$$\begin{aligned}
 & 2 \left[\begin{array}{l} \text{app-code} \\ \text{app-code} \end{array} \right. \\
 & \quad \text{for } (i = 1; i < n; i *= 2): \\
 & \quad \quad 2 \left[\begin{array}{l} \text{app-code} \\ \text{app-code} \end{array} \right. \\
 & \quad \quad \quad \left[\begin{array}{l} \text{for } (j = 0; j < n; j += 3): \\ \quad \text{app-code} \\ \quad \text{app-code} \end{array} \right. \\
 & \quad \quad \quad \quad \frac{2n}{3} + 3 \\
 & \quad \quad \quad \text{end for} \\
 & \quad \quad \left. \begin{array}{l} \text{app-code} \\ \text{end for} \end{array} \right]
 \end{aligned}$$

$$\log_2 n \left(\frac{2n}{3} + 3 \right)$$

$$\frac{2n}{3} \log_2 n + 3 \log_2 n + 2$$

$$\Theta(n \log n) = O(n \log n)$$

BIG-O ANALYSIS EXAMPLE

$n = 1000$ Matrix

$n = 3$

- Add Square Matrices

4	2	1
0	-3	4
5	6	2

 +

6	1	7
3	2	-1
4	6	2

 =

10	3	8
3	-1	3
9	12	4

BIG-O ANALYSIS EXAMPLE (CONT.)

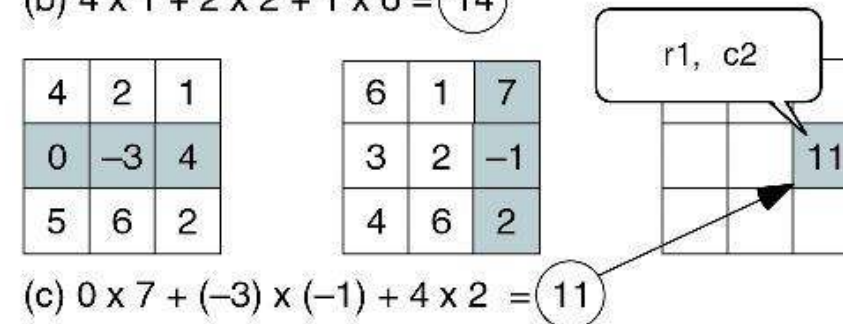
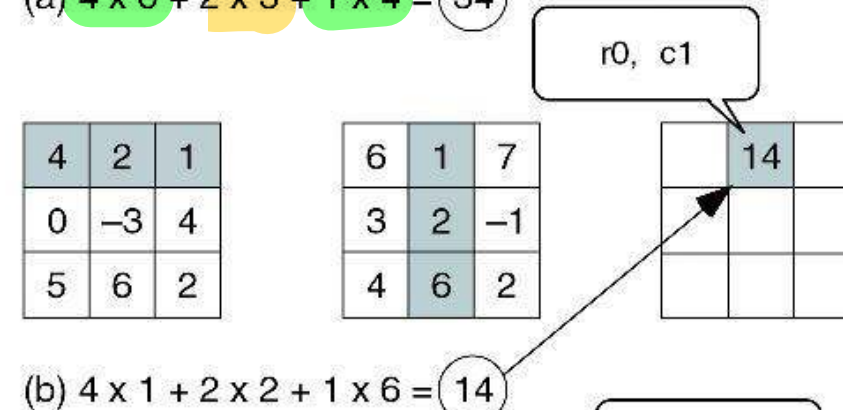
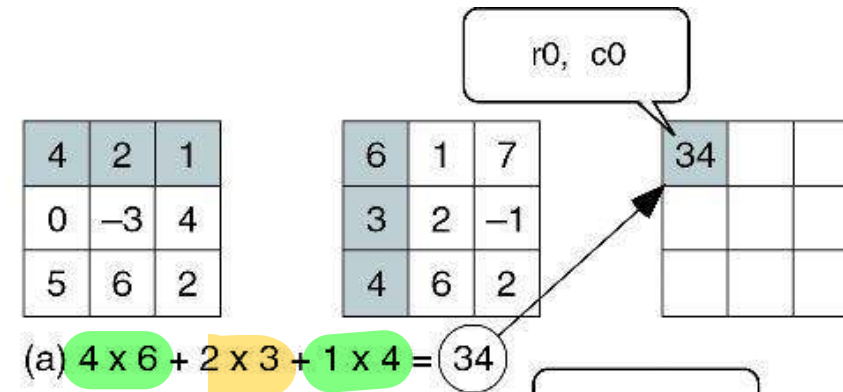
- Add Square Matrices

```
Algorithm addMatrix (matrix1, matrix2, size, matrix3)
Add matrix1 to matrix2 and place results in matrix3
  Pre  matrix1 and matrix2 have data
       size is number of columns or rows in matrix
  Post matrices added--result in matrix3
1 loop (not end of row) n rows
  1 loop (not end of column) n rows
    1 add matrix1 and matrix2 cells
    2 store sum in matrix3
  2 end loop
2 end loop
end addMatrix
```

$O(n^2)$

BIG-O ANALYSIS EXAMPLE (CONT.)

- Multiply Square Matrices



BIG-O ANALYSIS EXAMPLE (CONT.)

- Multiply Square Matrices

```
Algorithm multiMatrix (matrix1, matrix2, size, matrix3)
Multiply matrix1 by matrix2 and place product in matrix3
Pre  matrix1 and matrix2 have data
     size is number of columns and rows in matrix
Post matrices multiplied--result in matrix3
1 loop (not end of row)n
  1 loop (not end of column)n
    1 loop (size of row times)n
      1 calculate sum of
        (all row cells) * (all column cells)
      2 store sum in matrix3
    2 end loop
  2 end loop
2 end loop
3 return
end multiMatrix
```

$O(n^3)$

EXERCISE 1

- จงแสดง $f(n)$ และอัตราการเติบโตของโปรแกรมต่อไปนี้

Algorithm test1(n)

```
    for (i=0; i<n; i++)  
        for (j=0; j<5; j++)  
            app_code  
        end for  
        app_code  
    end for  
end test1
```

Handwritten analysis:

- A large bracket on the left of the code spans from the first `for` loop to the `end test1` line, labeled with n .
- A smaller bracket on the right of the code spans the inner loop body (the `app_code` line), labeled with 5 .
- Handwritten text to the right of the code shows the calculation: $n(10 + 1) = 11n$.
- Below that, the complexity is given as $O(n)$.

EXERCISE 2

- จงแสดง $f(n)$ และอัตราการเติบโตของโปรแกรมต่อไปนี้

Algorithm test2(n)

```

log2 n {
    for (i=1; i<n; i*=2)
        n/2 {
            for (j=0; j<n; j+=2)
                app_code
            end for
            2 {
                app_code
                app_code
            }
        }
    end for
end test2
```

$$\log_2 n \left(2 + \frac{n}{2} \right)$$

$$2 \log_2 n + \frac{n \log_2 n}{2}$$

$$O(n \log_2 n)$$